

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»**

**Факультет компьютерных наук
Основная образовательная программа
«Прикладная математика и информатика»**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**Программный проект на тему
Гибкий фреймворк распределённого глубинного
обучения в среде добровольных вычислений**

**Выполнил студент группы 216, 4 курса,
Миронов Сергей Дмитриевич**

**Руководитель ВКР:
Кандидат наук, Доцент, Каледин Максим Львович**

**Соруководитель ВКР:
Бакалавр, Соколовский Алексей Константинович**

Аннотация

Современные модели глубинного обучения выросли до размеров, при которых их сложно обучать на одном устройстве, а использование кластера часто оказывается дорогостоящим и требует от исследователей навыков работы с распределёнными системами, что усложняет научную работу в данной области. Для решения этих проблем предлагается STOILO (System for Task Orchestration of Independent Loosely-coupled Operations) — гибкий фреймворк распределённого исполнения задач, обеспечивающий выполнение Python-функций с произвольными зависимостями Python-окружения в парадигме добровольных вычислений на базе программного обеспечения BOINC. В частности, система удобна для распределённого глубинного обучения. В работе описана и обоснована архитектура системы, предоставлен исходный код и показано, как система может применяться для решения задач глубинного обучения. Основным результатом работы является создание удобного и применимого на практике фреймворка для выполнения произвольных Python-задач в сети, основанной на платформе BOINC. Предложенное решение может быть адаптировано для широкого круга прикладных проектов.

Abstract

Modern deep learning models have grown to a size where they are difficult to train on a single machine, while using a cluster can be expensive and requires researchers to have skills in distributed systems, making research in this area more difficult. To address these challenges, we propose STOILO (System for Task Orchestration of Independent Loosely-coupled Operations) — a flexible framework for distributed task execution that supports running Python functions with arbitrary Python-environment dependencies in a volunteer computing paradigm based on the BOINC middleware and is particularly well suited for distributed deep learning. We describe and justify the architecture of the system, provide its source code, and demonstrate how the system can be applied to deep learning tasks. The main outcome of this work is the creation of a convenient and practically applicable framework for executing arbitrary Python tasks within a network based on BOINC. The proposed solution can be adapted for a wide range of applied projects.

Ключевые слова

Распределённое глубинное обучение, BOINC, Python Master-Worker

Содержание

Аннотация	2
Abstract	2
1 Введение	5
2 Системы добровольных и grid вычислений	5
3 Системы машинного обучения на основе BOINC	7
4 Модель вычислений и хранения данных BOINC	9
5 Типы распределённых систем машинного обучения	11
5.1 Внутриузловой параллелизм	11
5.2 Межузловой параллелизм	11
5.3 Синхронность обновлений	13
5.4 Сосредоточение данных	13
6 Постановка задачи	14
7 Описание архитектуры системы	15
7.1 Пользовательская Python-библиотека	15
7.2 Серверные процессы	20
7.3 Приложения BOINC и спецификация Python-окружения	22
7.4 Межверсионный транспорт Python-вызовов	25
8 Машинное обучение в STOILO	27
8.1 Синхронный градиентный спуск	28
8.2 Частично синхронный градиентный спуск	30
9 Тестирование и эксплуатация	31
9.1 Средства развёртывания STOILO	31
9.2 Развёртывание сервера в облаке	32
9.3 Разнородные физические узлы	32
9.4 Прерываемые контейнерные узлы	33
9.5 Проверка обёрток для машинного обучения	33

10 Безопасность	35
10.1 Безопасность устройств добровольцев	35
10.2 Безопасность сервера и исследователя	36
10.3 Контроль доступа к планированию задач	36
11 Направления дальнейшей работы	37
11.1 Развитие обёрток для машинного обучения	37
11.2 Избыточная репликация вычислений	38
12 Заключение	39
A Физические устройства тестовой инсталляции	40
Список литературы	42

1 Введение

Машинное обучение, а особенно его направление — глубинное обучение, — является быстроразвивающейся и крайне перспективной областью компьютерных наук. Глубинное обучение помогает решать широкий спектр сложных задач: от распознавания объектов на изображениях и видео до генерации естественного текста и диагностики заболеваний по медицинским снимкам. Однако выдающиеся модели глубинного обучения часто обладают большими размерами: они содержат миллионы параметров и требуют выполнения значительного количества вычислительных операций. Например, ResNet-152 — современная нейросетевая архитектура для классификации изображений — имеет более 60 миллионов параметров и предполагает выполнение сотен миллиардов операций с числами с плавающей точкой (FLOPS) в процессе обучения [16]. Кроме того, такие модели требуют объёмных датасетов для обучения. Таким образом, модели, демонстрирующие высококачественные результаты, предъявляют серьёзные требования к вычислительным ресурсам. Ресурсов одной машины быстро становится недостаточно, а построение классических решений в виде on-premises или облачных кластеров является финансово затратным.

В рамках данного проекта мы предлагаем STOILO (System for Task Orchestration of Independent Loosely-coupled Operations) — систему распределённого машинного обучения, основанную на концепции добровольных вычислений — модели, в которой пользователи предоставляют недоиспользованные ресурсы своих устройств научным исследованиям из альтруистических соображений. Важным требованием к создаваемой системе является простота и гибкость в использовании: с её помощью должно быть возможно и удобно обучать произвольные модели глубинного обучения.

Исходный код системы опубликован в GitHub репозитории автора и доступен по ссылке <https://github.com/sermir2003/stoilo>.

2 Системы добровольных и grid вычислений

Добровольные вычисления (англ. *Volunteer Computing, VC*) — это модель распределённых вычислений, в которой владельцы персональных устройств предоставляют их недоиспользованные ресурсы научным проектам на безвозмездной основе. Устройства добровольцев обрабатывают отдельные задачи крупных исследований в фоновом режиме, формируя мощную распределённую систему. Такой подход позволяет существенно сократить расходы на вычислительные мощности и задействовать ресурсы, на которые не обращают внимания

другие вычислительные парадигмы [3].

Яркими примерами успешного применения добровольных вычислений являются проекты *SETI@home*, занимающийся анализом радиосигналов из космоса [4], *Rosetta@home*, предсказывающий трёхмерные структуры, моделирующий взаимодействия и проектирующий по заданным свойствам белки; результаты, полученные в рамках данного проекта, были использованы при разработке некоторых лекарств и вакцин [24], *Folding@home*, специализирующийся на моделировании процессов сворачивания белков и внёсший значительный вклад в исследование COVID-19 [25]. Перечисленные проекты привлекли сотни тысяч участников по всему миру и наглядно продемонстрировали практическую применимость добровольных вычислений в решении научных задач.

Модель добровольных вычислений тесно связана с концепцией grid-вычислений, направленной на объединение ресурсов нескольких организаций, например, институтов, с целью совместного решения сложных задач [15]. Основное отличие добровольных вычислений заключается в отсутствии административного контроля над вычислительными узлами, что порождает дополнительные вопросы безопасности и защиты от злонамеренного поведения участников. Кроме того, системы добровольных вычислений изначально рассчитаны на более высокую гетерогенность и широкую географическую распределённость узлов по сравнению с grid-системами. В некотором смысле добровольные вычисления представляют собой более общий класс распределённых систем, не делающий ряда предположений о свойствах вычислительной среды. Создание системы на основе добровольных вычислений выглядит крайне привлекательным: подобная архитектура универсальна и при необходимости может быть адаптирована как для использования ресурсов организаций — например, неактивно используемых компьютеров учебных классов университета, — так и для экономичной работы в облачных средах с использованием прерываемых виртуальных машин, стоимость которых на 70–90% ниже машин со стандартными гарантиями надёжности.

Существует несколько платформ для организации добровольных вычислений:

- *BOINC* (Berkeley Open Infrastructure for Network Computing) — универсальная и масштабируемая система, лежащая в основе множества успешных научных проектов [3].
- *Charity Engine* — коммерческая инициатива, использующая модифицированную версию BOINC для монетизации вычислительных мощностей и передачи вырученных средств на благотворительные нужды.
- *DreamLab* — мобильная платформа Vodafone Foundation, позволяющая задействовать вычислительные ресурсы смартфонов для медицинских исследований [22].

Среди этих платформ *BOINC* является наиболее технически зрелой и распространённой. Её ключевые преимущества включают:

- *Поддержку гетерогенных устройств и ускорителей* — выполнение задач на CPU, GPU (через OpenCL и CUDA), а также на мобильных устройствах.
- *Развитую инфраструктуру* — планирование заданий в гетерогенной среде, сбор и верификацию результатов, автоматическое управление ресурсами.
- *Высокую масштабируемость* — возможна успешная работа с сотнями тысяч узлов и миллионами заданий в день, существует база для горизонтального масштабирования.
- *Активную разработку и сообщество* — регулярные обновления, открытый исходный код и множество доступной справочной информации.

Фактически BOINC стала стандартом добровольных вычислений: подавляющее большинство современных проектов в этой области основаны именно на данной платформе [17]. Это делает выбор BOINC рациональным для создания новых систем добровольных вычислений, обеспечивая техническую зрелость, гибкость и широкую поддержку.

3 Системы машинного обучения на основе BOINC

Существует немало проектов на базе BOINC, связанных с машинным обучением. Ниже рассмотрены несколько наиболее интересных из них, с акцентом на универсальность и удобство их использования для инженеров машинного обучения.

MLC@Home — один из наиболее известных проектов машинного обучения, запущенный в 2020 году Университетом Мэриленда. Его целью было не обучение нейросетей для непосредственного применения, а исследование свойств весов обученных моделей. Добровольцы обучили множество небольших моделей, в результате чего был собран датасет MLDS [10]. Хотя проект дал ценные научные данные в области интерпретируемости моделей, он не предоставлял пользователю интерфейс для обучения произвольных нейросетей: архитектура модели была жёстко зафиксирована в C++ коде, и для её изменения требовалось изменение кода проекта.

Large Scale Evolution of Convolutional Neural Networks Using VC — статья 2017 года, в которой был использован алгоритм EXACT для эволюционного построения свёрточных нейросетей [14]. Добровольцы обучили различные варианты архитектур, постепенно

улучшая их. BOINC позволил провести масштабный перебор моделей, однако, как и в случае с MLC@Home, система была жёстко ориентирована на одну задачу и не предусматривала обучение произвольных моделей, заданных пользователем.

DistributedDataMining (dDM) — проект 2013 года, предлагающий выполнение RapidMiner поверх BOINC [21]. RapidMiner является графической средой для построения пайплайнов анализа данных. В dDM добровольцы через BOINC исполняли сценарии RapidMiner, а результаты собирались на сервере. Такой подход позволил распределённо решать ресурсоёмкие задачи прогнозирования рыночной стоимости акций, анализа медицинских данных и оптимизации моделей машинного обучения. Однако использование системы сопровождалось рядом трудностей: пользователь должен был вручную создать и экспортировать RapidMiner process в формате XML, разумно разбить вычисление на независимые задачи (jobs), подготовить отдельные input/output файлы для каждой задачи и запустить вычисления через BOINC wrapper-программу для RapidMiner. Фактически интегрированного с runtime BOINC интерфейса создано не было, вместо этого предлагалось вручную портировать процессы из RapidMiner на BOINC.

Distributed Deep Learning Using Volunteer Computing-Like Paradigm — статья 2019 года, в которой предложена архитектура распределённого глубинного обучения на базе BOINC [5]. Авторы реализовали обучение в парадигме Data Parallelism с Parameter Server и предложили схему асинхронного обновления градиентов VC-ASGD. Они обучили несколько нейросетей, продемонстрировав потенциал распределённого обучения в добровольной среде. Однако исходный код системы не был опубликован. Авторы заявили о создании гибкого интерфейса, в котором достаточно средствами TensorFlow определить модель, задать функцию потерь и указать датасет, после чего обучение произойдёт автоматически. Тем не менее полноценного инструмента, готового для конечного применения, предъявлено не было.

Анализ существующих проектов показывает, что платформа BOINC обладает высоким потенциалом для решения задач машинного обучения, позволяя масштабировать ресурсоёмкие вычисления за счёт распределения нагрузки между добровольными участниками. Однако все рассмотренные системы либо ориентированы на выполнение узкой фиксированной задачи, либо требуют значительных технических усилий от пользователя, включая написание кода на C++ и ручную адаптацию процессов под модель BOINC. Существенным достижением в области универсализации является работа Atre и др. [5], предложившая архитектуру обучения моделей на основе TensorFlow, однако полноценный инструмент для практического применения опубликован не был.

4 Модель вычислений и хранения данных BOINC

BOINC является программным обеспечением с открытым исходным кодом для разработки проектов добровольных вычислений [2, 3]. Система на основе BOINC состоит из совокупности серверных процессов (демонов), базы данных MySQL или MariaDB и клиентского программного обеспечения для выполнения заданий на устройствах добровольцев.

Поскольку обычно устройства волонтеров не имеют публичных IP адресов и находятся за NAT, модель BOINC предполагает, что все коммуникации происходят по инициативе клиентов: клиент устанавливает соединение с сервером, информирует о своих ресурсах, запрашивает новые задания и, возможно, доставляет результаты выполненных заданий.

Серверные демоны — это процессы, выполняющие конкретные логические функции и взаимодействующие с базой данных и файловой системой сервера. К ним относятся:

- **work_generator** — генерирует новые задания, формирует входные данные, оценивает вычислительную сложность и задаёт требования к клиентам;
- **scheduler** — отвечает на сетевые запросы клиентов, принимает результаты и исходя из доступных ресурсов клиента, истории взаимодействия с ним, а также с учётом списка имеющихся задач, их требований к клиентам, предполагаемой сложности и заданных параметров, принимает решение о назначении новых задач;
- **transitioner** — отслеживает состояние заданий и выполняет необходимые действия согласно конечному автомату проекта, например, инициирует новые вычисления, если для некоторого задания собрано недостаточно корректных результатов;
- **validator** — проверяет корректность результатов, выявляет ошибки и зловерное поведение; для защиты от злонамеренного поведения BOINC применяет репликацию вычислений с последующими сравнениями результатов, выполняемыми валидатором;
- **assimilator** — обрабатывает валидированные результаты, может, например, сохранять их в базу данных или инициировать последующую обработку;

и некоторые другие служебные, например, **feeder**, **file_deleter**, **db_purge**.

Важно, что разработчикам проекта на базе BOINC как правило достаточно реализовать только три компонента: **work_generator**, **validator** и **assimilator** с бизнес-логикой проекта. Остальные демоны предоставляются BOINC.

Помимо серверной части, требуется реализовать код, который будет запускаться клиентом BOINC на узлах добровольцев. В терминологии BOINC различают два понятия: *app* и

app versions. *app* — это логически индивидуальное приложение, а *app versions* — конкретная семантическая версия приложения для конкретной архитектуры. В качестве иллюстрации приведём пример: код, вычисляющий интеграл заданной функции на запрошенном отрезке методом трапеций, — это приложение, так как речь идёт только о логике программы; скомпилированное приложение версии 1.0 для операционной системы Windows, архитектуры x86_64 — это версия приложения. Разработчик проекта обязан самостоятельно подготовить версии приложений для всех целевых платформ. В старых выпусках BOINC версия приложения должна была представлять из себя один исполняемый файл, однако в настоящее время BOINC поддерживает комплектацию версий приложений из нескольких файлов, что позволяет запускать приложения через обёртки (wrappers) или внутри виртуальной машины или Docker контейнера.

Что касается управления данными, классическая модель BOINC предполагает взаимодействие клиентов исключительно с центральным сервером. Горизонтальное распределение данных возможно: файлы могут храниться на дополнительных серверах или в отдельных базах данных для повышения масштабируемости, однако BOINC изначально не предусматривает надёжное хранение данных на клиентах или общение клиентов между собой.

Фундаментальным понятием системы является файл: каждый файл в BOINC неизменяем и идентифицируется уникальным именем. В задании указывается список необходимых файлов, а сервер и клиент автоматически управляют их доставкой, хранением и удалением. Влиять на эти процессы можно, например, такими настройками, как *Sticky flag* — не удалять файл после выполнения задания или *Locality scheduling* — учитывать уже загруженные файлы при планировании новых заданий, что позволяет назначать задачи на узлы, где уже присутствует некоторый необходимый объёмный файл.

Проблема масштабируемости передачи данных была рассмотрена в работе Costa и др. [11], где предлагалось использовать протокол BitTorrent для распределённой доставки больших файлов между клиентами. Решение позволило существенно снизить нагрузку на сервер, однако потребовало модификации клиентского программного обеспечения и оказалось не слишком эффективным при малом числе участников в сети и при малых размерах файлов.

Альтернативную архитектуру предложили Foster и др. [1], внедрив систему хранения данных на подмножестве добровольных узлов (Storage Agents). Такой подход улучшал масштабируемость для задач с большими объёмами данных и высокой географической распределённостью, но требовал дополнительной координации (NAT traversal) и надёжного контроля целостности информации, что в значительной мере усложняло архитектуру проекта.

5 Типы распределённых систем машинного обучения

Распределённое машинное обучение может проводиться с использованием различных стратегий. Руководствуясь обзорной статьёй Verbraeken и др.[23] и метаанализом Ben-Nun и др.[6], основанным на анализе 252 систем и публикаций, описывающих различные аспекты распределённого и параллельного машинного обучения, можно выделить следующие степени свободы архитектур систем машинного обучения. Для каждой из этих степеней свободы мы рассмотрим возможные варианты реализации, выберем наиболее подходящий для парадигмы добровольных вычислений и системы на базе middleware BOINC и обоснуем сделанный выбор.

5.1 Внутриузловой параллелизм

Single-machine Parallelism — ускорение вычислений за счёт оптимального использования параллелизма и ускорителей на уровне одного узла: многопоточность и многопроцессорность CPU, TPU и GPU.

Разработка собственных низкоуровневых или математических оптимизаций выполнения арифметических операций, возникающих в процессе машинного обучения, является крайне сложной задачей и выходит за рамки данной работы. А без глубоких оптимизаций арифметических действий реализовать собственную среду исполнения, сопоставимую по производительности со средами исполнения PyTorch или TensorFlow, не представляется возможным.

Поэтому для выполнения операций на уровне отдельного узла наиболее разумным представляется использование существующих программных средств, нежели реализация собственных. В частности, таким механизмом может служить запуск пользовательских Python-функций и передача их возвращаемых значений в качестве результатов исполнения задач. Преимущество такого подхода особенно заметно при использовании в окружении выполнения Python библиотек вроде NumPy, PyTorch или TensorFlow, в которых уже реализованы развитые оптимизации вычислительных операций.

5.2 Межузловой параллелизм

Multi-machine Parallelism — способы распределения работы между несколькими машинами. Возможны следующие подходы:

Data Parallelism — обучение одной и той же модели на разных подмножествах данных,

распределённых между узлами. Этот метод позволяет обрабатывать объёмные датасеты, которые не помещаются на одной машине, при условии, что модель может быть реплицирована на каждый узел.

Model Parallelism — разделение вычислительного графа модели между несколькими машинами. Такой подход упрощает обучение больших моделей, однако передача данных прямого и обратного распространений в распределённой среде может вносить существенные накладные расходы.

Pipelining — стратегия конвейерной обработки: каждый узел выполняет свою часть последовательности операций над входными данными. Благодаря параллельному выполнению нескольких операций обучения, конвейеризация позволяет повысить загрузку оборудования, использование вычислительных мощностей, но усложняет управление зависимостями и требует надёжной синхронизации.

Hybrid Parallelism — произвольная комбинация перечисленных выше методов. Эти подходы не являются взаимоисключающими и могут применяться совместно в зависимости от задачи и инфраструктуры.

Для среды добровольных вычислений наиболее разумным вариантом параллелизма между узлами представляется Data Parallelism, так как он предъявляет наименее жёсткие требования к задержке и пропускной способности коммуникации между узлами.

В любой модели параллелизма перед очередным шагом обучения, например, шагом градиентного спуска, необходимо получить результаты частичных вычислений с участвующих узлов. Но в параллелизме по данным возможно совершить шаг, собрав результаты лишь для ограниченной доли узлов — это крайне естественный для машинного обучения подход: использовать лишь часть датасета.

В случае параллелизма модели, напротив, неочевидно, каким образом можно агрегировать результаты вычисления разных частей модели, полученные в разных эпохах обучения, для последующего обновления её параметров. Следует отметить, что это не является принципиально невозможным, в зависимости от конкретной задачи могут быть эффективны разные эвристики, но в общем случае наиболее перспективным представляется параллелизм по данным. Pipelining и комбинированные подходы требуют ещё более тесной синхронизации, стоимость которой в географически сильно распределённой среде крайне высока.

5.3 Синхронность обновлений

Synchronization — стратегии синхронизации градиентов и весов в процессе выполнения градиентного спуска. Возможные варианты:

Synchronous — каждый шаг градиентного спуска выполняется строго после сбора и усреднения градиентов от всех вычислительных узлов.

Stale-Synchronous — допускается использование устаревших градиентов в пределах фиксированного “запаздывания”. Это снижает требования к синхронизации при сохранении теоретических гарантий сходимости алгоритма оптимизации.

Asynchronous — каждый шаг спуска выполняется на основе градиентов, доступных на данный момент времени. Такой подход уменьшает накладные расходы на синхронизацию, но может негативно сказаться на скорости и качестве сходимости. Вообще говоря, гарантий сходимости у подобного градиентного спуска обычно нет.

Nondeterministic Communication — модель, при которой передача градиентов и параметров модели происходит между узлами случайным образом в сложной топологии без строгой синхронизации.

Стратегия синхронизации частичных результатов участников вычислений — это компромисс между сходимостью метода и накладными расходами коммуникации и синхронизации. В добровольной вычислительной сети эти накладные расходы особенно высоки, поэтому наиболее сбалансированным выглядит Stale-Synchronous подход.

Впрочем, как будет показано далее, в предлагаемой нами архитектуре может быть использована любая из стратегий, так как логика синхронизации реализована в программном обеспечении, работающем на стороне инженера машинного обучения, а не на сервере или устройстве добровольца.

5.4 Сосредоточение данных

Centralization — способы организации хранения и обновления параметров модели. Возможны следующие архитектуры:

Parameter Server (PS) — один или несколько выделенных серверов, которые хранят все параметры модели, принимают градиенты от рабочих узлов и отправляют обновлённые веса обратно. Такая архитектура упрощает координацию, но может стать узким местом при высокой нагрузке.

Sharded PS — распределение пространства параметров между несколькими серверами.

Каждый сервер отвечает только за часть параметров (шард), что позволяет уменьшить нагрузку на сеть и повысить масштабируемость.

Hierarchical PS — многоуровневая организация серверов параметров: рабочие узлы взаимодействуют с локальными серверами, которые в свою очередь координируются с центральными серверами. Такая схема снижает сетевые затраты и улучшает масштабируемость в больших кластерах.

Decentralized — рабочие узлы напрямую обмениваются градиентами и параметрами друг с другом, без выделенного сервера. Обновления могут происходить посредством коллективных операций, например, AllReduce, что устраняет единую точку отказа, но требует эффективной синхронизации.

В разделе 4 рассматривались различные способы расширения классической архитектуры BOINC, включая горизонтальное масштабирование сервера [3], построение хранилища на основе устройств добровольцев [1, 11] и схемы взаимодействия между добровольными узлами [1], для чего обычно требуется NAT traversal. Однако в большинстве сценариев распределённого машинного обучения такие механизмы вряд ли способны дать существенный выигрыш в производительности: они усложняют архитектуру и при этом требуют большей предсказуемости поведения добровольцев, усложняя механизмы верификации результатов. Поэтому более оправданным выбором является классическая централизованная схема BOINC, в которой Parameter Server размещается на стороне сервера системы.

Таким образом, для системы, работающей в парадигме добровольных вычислений, наиболее оправданной является архитектура, сочетающая параллелизм по обучающим данным (Data Parallelism), синхронизацию параметров модели с ограниченным запаздыванием (Stale-Synchronous), централизованный сервер параметров (Parameter Server) и делающая упор на существующие программные экосистемы локального исполнения.

6 Постановка задачи

Как показал обзор существующих решений, в настоящее время отсутствует универсальная и готовая к практическому применению система распределённого машинного обучения на базе платформы BOINC. Имеющиеся проекты либо ориентированы на решение узкоспециализированных задач, либо требуют от пользователей значительных усилий по

интеграции моделей, включая необходимость ручной адаптации к инфраструктуре и написания низкоуровневого кода на C/C++. В результате существующие системы оказываются малоприспособленными для широкого круга инженеров машинного обучения, ожидающих простого и привычного интерфейса для построения и обучения моделей.

Целью настоящей работы является разработка системы распределённого машинного обучения на базе middleware BOINC, сочетающей вычислительные возможности среды добровольных вычислений с удобством современных фреймворков машинного обучения. Основные требования к системе включают универсальность в отношении типов моделей и задач, удобство использования, не требующее вмешательства в низкоуровневые компоненты, а также возможность эффективного масштабирования в распределённой среде добровольных вычислений.

7 Описание архитектуры системы

В данном разделе описывается архитектура STOILO — системы на базе BOINC для выполнения Python-функций на устройствах участников сети добровольных вычислений. Система позволяет пользователю настраивать окружение исполнения Python, включая версию интерпретатора, а также набор и версии библиотек из репозитория PyPI. В следующем разделе 8 мы продемонстрируем гибкость этих абстракций при решении задач машинного обучения: интерфейс практически не ограничивает ни устройство модели, ни стратегию распределения вычислений. Здесь же мы ограничимся простыми примерами, иллюстрирующими ключевые принципы работы системы.

Система состоит из трёх основных компонентов: пользовательской Python-библиотеки `stoilo`, серверной инсталляции STOILO на базе BOINC и версий приложений для клиентов BOINC `rt_*`. В следующих подразделах последовательно рассматриваются роли этих компонентов в системе, их устройство и взаимодействие в процессе выполнения задач.

7.1 Пользовательская Python-библиотека

Пользователем или исследователем в терминологии STOILO будем называть человека, формирующего вычислительные задачи и получающего результаты их выполнения. Взаимодействие пользователя с системой осуществляется через клиентскую Python-библиотеку `stoilo`, она размещена в директории `python_lib` GitHub-репозитория автора, имеет стандартную для Python-пакетов файловую структуру и устанавливается в пользовательское

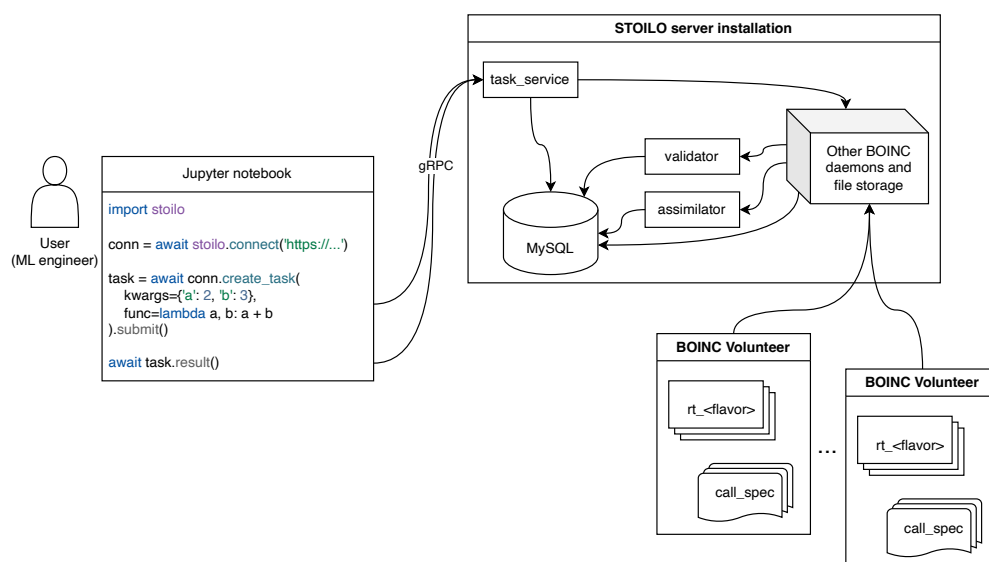


Рис. 7.1: Обзорная диаграмма STOILO.

окружение с помощью пакетного менеджера `pip`: `pip install -e python_lib`. Установку рекомендуется выполнять внутри виртуального окружения. В перспективе библиотека может быть опубликована в одном из индексов пакетов, например, в Python Package Index (PyPI).

```

1 import stoilo
2
3 # Assumes authenticated and encrypted access to the server endpoint
4 conn = await stoilo.connect('localhost:57010')
5
6 # synchronous non-blocking creation
7 task = conn.create_task(
8     kwargs={'a': 2, 'b': 3},
9     func=lambda a, b: a + b
10 )
11 # task has not been submitted to the server yet
12 print(task.task_id is None) # True
13
14 # asynchronous submission, will return fairly quickly
15 task = await task.submit()
16 # task has been submitted and the task_id has been assigned
17 print(task.task_id) # some uuid
18
19 # asynchronous waiting for the result, takes some time to finish
20 result = await task.result()
21 print(result) # 5
22
23 # or for short
24 await conn.create_task(
25     kwargs={'a': 2, 'b': 3},
26     func=lambda a, b: a + b
27 ).result() # 5

```

Листинг 1: Пример создания и отправки задачи и получения результата.

Для начала пользователю необходимо установить соединение с сервером; вопросы авторизации и аутентификации подробнее рассматриваются в разделе 10.

Асинхронная функция `connect` создаёт объект класса `Connection`, через который пользователь может формировать задачи. Для создания новой задачи используется метод `Connection.create_task(...)`, обязательными аргументами которого являются `func` и `kwargs`. Идентификатор `func` задаёт вызываемый Python-объект, принимающий произвольный набор именованных аргументов и возвращающий значение произвольного типа. Идентификатор `kwargs` содержит значения, с которыми этот объект будет вызван при исполнении на узле участника добровольной сети. Логически это словарь, отображающий имена аргументов в их значения. Вопросы сериализации входных данных задачи и возвращаемого объекта рассматриваются в подразделе 7.4.

Метод `Connection.create_task(...)` является неблокирующим и немедленно возвращает объект класса `StagedTask`. На данном объекте можно вызвать асинхронный метод `.submit()`, который отправит задачу на сервер и вернёт экземпляр класса `SubmittedTask`, на котором в свою очередь можно асинхронно ожидать завершения вычисления с помощью метода `.result()`. Данный вызов завершится строго после выполнения задачи, что в условиях среды добровольных вычислений может занимать значительное время.

Пользователь библиотеки не обязан всё время обработки задачи поддерживать Python процесс на своём компьютере в активном состоянии, вместо этого он может надёжно сохранить атрибут `.task_id` объекта `SubmittedTask` и остановить Python интерпретатор или ядро Jupyter Notebook-a. После возвращения к работе пользователю понадобится лишь создать новое соединение и с помощью метода `Connection.restore_task(task_id)` получить соответствующий экземпляр `SubmittedTask`. Впрочем, для упрощения интерфейса, экземпляры класса `StagedTask` также имеют метод `.result()`, выполняющий внутри себя `.submit().result()`. Такой метод удобно использовать для задач с небольшим ожидаемым временем выполнения.

В любом случае по завершении обработки задачи метод `.result()` вернёт объект составного типа `TaskResult=Union[Any, UserError, SystemError]` по следующим правилам:

1. Если функция `func` завершилась успешно необходимое число раз, а возвращённые значения прошли валидацию, то есть в том числе признаны эквивалентными, то метод `.result()` вернёт одно из эквивалентных возвращённых значений функции.
2. Если из функции `func` выбрасывались исключения, и был собран кворум её запусков, результатами которых стали исключения, объект любого из этих исключений возвращается в экземпляре класса `stoilo.low_level.UserError`, наследника класса `Exception`.
3. Если собрать кворум запусков задачи с эквивалентным результатами по любым при-

чинам не удалось, например, если был достигнут лимит запусков задачи, `.result()` вернёт объект класса `stoilo.low_level.SystemError`, наследника класса `Exception`.

Метод `Connection.create_task(...)` помимо `func` и `kwargs` принимает ещё несколько аргументов, задающих индивидуальные настройки Python-задачи, а именно:

- `init_valid_func` типа `Callable[[Any], bool]` — функция первичной валидации, которая будет независимо применена к каждому возвращаемому значению функции `func`. По умолчанию `lambda _: True`.
- `compare_valid_func` типа `Callable[[Any, Any], bool]` — функция сравнительной валидации, которая будет применяться к паре возвращённых функцией `func` значений с целью установления их эквивалентности. По умолчанию `lambda x, y: x == y`.
- `redundancy_options` типа `task_service_pb2.RedundancyOptions` — объект, задающий параметры вычислительной избыточности, необходимой для валидации результатов и защиты от злонамеренного поведения со стороны добровольцев. Пользователь может создать его с помощью функции `stoilo.low_level.redundancy.CreateOptions`, принимающей некоторые параметры, выводящей недостающие и выбрасывающей исключение в случае некорректной совокупности параметров.
- `flavor` типа `str` — строка, характеризующая Python окружение исполнения. Механизм задания окружений исполнения и его связь с приложениями и версиями приложений BOINC рассматриваются в подразделе 7.3.

```
1 def quadratic_roots_harmonic_mean(a, b, c):
2     d = b ** 2 - 4 * a * c
3     if d < 0:
4         return None
5     x_1, x_2 = (-b - d ** 0.5) / (2 * a), (-b + d ** 0.5) / (2 * a)
6     return 2 / (1 / x_1 + 1 / x_2)
7
8 await conn.create_task(
9     kwargs=dict(a=1, b=-3, c=2),
10    func=quadratic_roots_harmonic_mean,
11    init_valid_func=lambda res: res is None or isinstance(res, float),
12    compare_valid_func=lambda x, y: (
13        x is None and y is None
14        or x is not None and y is not None and abs(x - y) < 1e-6
15    ),
16    redundancy_options=stoilo.low_level.redundancy.CreateOptions(
17        min_quorum=3,
18        max_total_results=5
19    ),
20 ).result()
```

Листинг 2: Пример настройки валидаторов и параметров репликации.

Библиотека `stoilo` взаимодействует с серверной частью системы посредством gRPC-вызовов, составляющих сервис `task_service`. Вызов `CreateTask` выполняется в жизненном цикле задачи один раз внутри метода `StagedTask.submit()`. Он передаёт на сервер исполняемую функцию, аргументы и параметры запуска задачи. Сервер регистрирует вычисление и возвращает присвоенный ему идентификатор `task_id`. Этот идентификатор используется в последующих обращениях к серверу: вызов `PollTask` позволяет получить текущий статус обработки, а в случае завершения вычисления — результат или сведения об ошибке. Метод `SubmittedTask.result()` совершает вызовы `PollTask` в цикле, экспоненциально увеличивая временной интервал между последовательными запросами (exponential backoff).

Асинхронность методов пользовательского интерфейса позволяет применять стандартные средства `asyncio`: `gather`, `wait`, `wait_for`, `Condition`, `Semaphore`. С их помощью пользователь может конструировать сложные графы зависимостей между задачами, в том числе проводить вычисления параллельно на многих узлах. Пример дан в листинге 3.

```
1 import asyncio
2
3 async def integrate(conn, f: Callable[[float], float], a: float, b: float) -> float:
4     def worker(x_0, x_n, n, f):
5         h = (x_n - x_0) / n
6         return h * (0.5 * (f(x_0) + f(x_n)) + sum(f(x_0 + j * h) for j in range(1, n)))
7
8     n_tasks = 10
9     n_steps_per_task = 1000
10    subseg_size = (b - a) / n_tasks
11    tasks = [
12        conn.create_task(
13            kwargs={
14                'x_0': a + i * subseg_size,
15                'x_n': a + (i + 1) * subseg_size,
16                'n': n_steps_per_task,
17                'f': f,
18            },
19            func=worker,
20            redundancy_options=stoilo.low_level.redundancy.TRIVIAL_OPTIONS,
21        )
22        for i in range(n_tasks)
23    ]
24    partial_integrals = await asyncio.gather(
25        *(task.result() for task in tasks)
26    )
27    return sum(partial_integrals)
28
29
30 await integrate(conn, lambda x: x * math.sin(x), 0, math.pi) # approximately pi
```

Листинг 3: Пример запуска параллельно исполняющихся задач.

Описанная выше модель общения с сервером проста, устойчива к разрыву соединения, падению клиента (если все `task_id` надёжно сохранены) и даже падению сервера, где состо-

яние обработки задачи транзакционно поддерживается в базе данных, надёжность которой можно повышать классическими средствами. Но самое главное — данная модель не требует от пользователей обладания публичным IP адресом. Часто исследователи работают на своих персональных компьютерах и скрыты за NAT, поэтому все взаимодействия библиотеки с сервером совершаются по инициативе библиотеки.

7.2 Серверные процессы

Автором данной работы STOILO предлагается не как централизованный глобальный сервис, а как система, разворачиваемая для конкретного исследовательского объединения, например, в рамках института или лаборатории. Пользовательская библиотека `stoilo` может взаимодействовать с любым развёрнутым сервером STOILO при наличии прав доступа, а версии приложений BOINC генерируются единообразно и могут использоваться в разных проектах STOILO. Однако для практического применения системы необходим оператор инсталляции — организация, которая привлечёт вычислительные ресурсы и распределит доступ к ним между пользователями.

Роль такого оператора может выполнять, например, научная лаборатория университета, которая, пользуясь своим авторитетом, сможет привлекать добровольцев со всего мира и на этой основе формировать вычислительный кластер для исследовательских нужд своих сотрудников. Другой сценарий — инсталляция на уровне высшего учебного заведения, использующая недозагруженные ресурсы компьютеров учебных классов для выполнения научных работ. Далее такое развёртывание будем называть инсталляцией исследовательского проекта на базе STOILO.

Инсталляция проекта не требует от оператора значительных усилий. Директория `server/deploy` GitHub-репозитория автора содержит всё необходимое для развёртывания серверной части: `Dockerfile`, `entrypoint.sh` и другие конфигурационные файлы. Они позволяют одной командой собрать и запустить контейнер, который самостоятельно установит соединение с базой данных, создаст табличную и файловую структуру проекта, зарегистрирует приложения, запустит необходимые процессы и развернёт веб-интерфейсы системы, включая панель администратора. Развёртывание собственного сервера STOILO подробно рассматривается в разделе [9](#).

Поскольку платформа BOINC уже использует базу данных MySQL/MariaDB для хранения служебной информации, в разработанной системе также используется MySQL-совместимая семантика доступа к БД, что позволяет работать на одном экземпляре базы

данных. Это полезно, если масштабы проекта не накладывают жёстких требований на поддержку БД, а перегружать сервер большим числом архитектурных компонентов не хочется.

Серверная часть включает в себя реализацию трёх основных BOINC демонов:

- **task_service** он же **work_generator** — принимает gRPC-соединения от пользователей. На запрос **PollTask** возвращает данные из БД таблицы **task_data**. На запрос **CreateTask** — генерирует уникальный идентификатор **task_id**, добавляет новую строку в таблицу **task_data** и создаёт BOINC сущность *workunit*, в рамках которой пользовательская Python-задача будет планироваться, выполняться и валидироваться.
- **validator** — выполняет проверку результатов вычислений по следующему алгоритму:
 - Для поступивших результатов, полученных в рамках некоторого *workunit*, достаёт из таблицы **task_data** функции **init_valid_func** и **compare_valid_func**, переданные пользователем во время создания задачи.
 - При первичной валидации (initial validation), **validator** одобряет результат, если он является **UserError** или если функция **init_valid_func** от данного результата возвращает **True**.
 - При сравнительной валидации (comparative validation) **validator** относит два результата к одному классу эквивалентности, если или они оба **UserError**, или оба не **UserError** и функция **compare_valid_func** от них вернула **True**.

Подобное лояльное отношение к исключениям в пользовательском коде вызвано тем, что задача валидации — защитить пользователя от некорректных результатов со стороны добровольцев, полученных, например, вследствие злонамеренного поведения. Однако выброс исключения из **func** вполне может являться корректным результатом исполнения функции. При этом допускается, что вызвавшая исключение ошибка на разных узлах может приводить к разным исключениям, поэтому все исключения считаются эквивалентными результатами. Мы рекомендуем пользователям писать такие функции **func**, из которых исключения не выбрасываются, хотя система готова к ошибкам исполнения пользовательского Python-кода.

Предложенный интерфейс позволяет выполнять произвольные пользовательские функции валидации, отвечающие бизнес-логике задачи, простой пример представлен в листинге 2. В качестве более сложного примера предположим, что пользователь в качестве результата возвращает из функции словарь, отображающий имя слоя модели

глубинного обучения в тензор градиентов. Тогда функция первичной валидации может проверять соответствие типам и размерностям, а сравнительной — сравнивать тензоры поэлементно, допуская зафиксированную пользователем погрешность в разности вещественных чисел.

- **assimilator** — принимает валидированные результаты вычислений и сохраняет их в таблицу `task_data`, откуда впоследствии они будут извлечены демоном `task_service` при ответе на gRPC-запрос `PollTask`.

Иллюстрация взаимосвязей между компонентами системы представлена на рис. 7.1.

7.3 Приложения BOINC и спецификация Python-окружения

В терминологии BOINC задания назначаются на *приложения* (*app*), а на устройствах волонтеров запускаются версии приложений (*app version*), собранные под данную конкретную операционную систему и архитектуру устройства добровольца.

Мы хотим предоставить исследователям возможность использовать в функциях `func` внешние Python библиотеки — модули, не входящие в стандартный дистрибутив Python, например, `numpy` или `pytorch`. Это желание сталкивается с рядом вызовов:

1. Каждая внешняя библиотека имеет собственное версионирование. Необходимо дать пользователям возможность управлять составом и версиями библиотек, которые будут доступны для использования в `func`.
2. Мы не можем упаковать абсолютно все библиотеки во всех версиях в BOINC приложение — файл версии приложения получится нереалистично большим.
3. В модели BOINC обмен данными с сервером реализует BOINC-клиент, а доступ версии приложения к сети может быть справедливо ограничен. Это не позволяет дозагрузить на узел необходимую библиотеку во время выполнения задачи.

Помимо пользовательских библиотек, необходимо следить за версиями интерпретатора Python и модулей, которые используются для сериализации Python-объектов, передаваемых пользователем серверу, а сервером — вычислительным узлам.

Мы предлагаем следующее решение: отдельное BOINC приложение для каждого набора зависимостей. Разумно ожидать, что на практике исследователям понадобится крайне

ограниченный набор различных конфигураций Python-окружения — в пределах 10–20 уникальных экземпляров со своими комбинациями библиотек фиксированных версий. И обновляться список конфигураций будет сравнительно редко, например, не чаще 1–2 раз в месяц.

Администраторы BOINC сервера имеют инструменты для добавления новых приложений и версий приложений в любой момент времени без необходимости останавливать сервер проекта. Благодаря этому механизму администраторы инсталляции STOILO смогут подготавливать новые приложения с необходимыми окружениями исполнения Python в соответствии с потребностями проекта.

Для описания конфигурации среды исполнения Python мы предлагаем JSON-формат, пример которого представлен в листинге 5. От канонизированного представления JSON-спецификации будем вычислять хеш и использовать его в качестве идентификатора конфигурации среды исполнения, назовём его словом *flavor*.

```
1 {  
2   "python_version": "3.12.3",  
3   "cloudpickle_version": "3.1.1",  
4   "requirements": [  
5     "numpy==2.2.5",  
6     "torch==2.7.0",  
7     "torchvision==0.22.0"  
8   ],  
9   "modules": [  
10    "numpy",  
11    "torch",  
12    "torchvision"  
13  ]  
14 }
```

Листинг 4: Пример спецификации среды исполнения Python. Поскольку имена дистрибутивов и модулей верхнего уровня совпадают, данный пример выглядит вырождено, но в общем случае имена библиотек и моделей могут различаться.

Строго говоря, способ генерации *flavor* не имеет принципиального значения, нам лишь нужен был строковый идентификатор, с которым система будет работать далее. Поэтому формат JSON-спецификации можно менять, а выбор алгоритма хеширования не влияет на логику системы. Тем самым обеспечивается прямая и обратная совместимость: главное избегать коллизий *flavor*.

Приложение BOINC, выполняющее функцию **func** с аргументами **kwargs** в фиксированной среде исполнения Python, будем называть *вариантом исполнения*. Идентификатор варианта исполнения формируется на основе соответствующего *flavor* и имеет вид **rt_<flavor>** (от англ. runtime).

Версию приложения BOINC, соответствующую фиксированному варианту исполнения для конкретной платформы, будем называть *сборкой исполнения*. Идентификатор сбор-

ки строится по шаблону `rt_<flavor>_<version>_<platform>`, где `platform` — имя платформы в реестре BOINC, а `version` — семантическая версия, используемая для выпуска обновлений исполняемого файла, например, при исправлении ошибок или внесении улучшений.

В GitHub-репозитории автора данной работы подготовлен минимальный набор инструментов для сборки исполнения. Базовая версия исполнителя функций находится в директории `workers/runner/`, примеры JSON-спецификаций конфигураций среды лежат в `workers/devops/flavors_spec/`, а скрипт `workers/devops/rt_builder.py` выполняет сборку исполнения для той платформы, на которой производится его запуск. Подобное ограничение обусловлено тем, что кросс-компиляция Python-программ сопряжена с серьёзными трудностями и корректность работы полученных бинарных файлов на целевой платформе гарантировать невозможно. Наиболее надёжным и простым решением является организация CI процесса с запуском `rt_builder.py` внутри виртуальных машин, соответствующих требуемым платформам. Такой процесс может быть реализован, например, в GitHub Actions.

```
1 ./rt_builder.py \  
2   --runtime-spec torch_baseline.json \  
3   --src ../runner \  
4   --base-python /Users/sermir2003/.pyenv/versions/3.12.3 \  
5   --version 1.0 \  
6   --platform arm64-apple-darwin
```

Листинг 5: Пример сборки исполнения для macOS с архитектурой Arm64.

Скрипт не устанавливает интерпретатор Python необходимой версии самостоятельно, путь к нему должен быть передан в аргументах командной строки. Это связано с тем, что способ установки интерпретатора определяется платформой сборки и описывать их в скрипте малоосмысленно. Вместе с тем данная процедура не является сложной, например, в macOS произвольную версию Python можно легко установить с помощью утилиты `pyenv`.

После передачи интерпретатора скрипт далее выполняет все шаги автоматически: создаёт новое виртуальное окружение, устанавливает библиотеки согласно спецификации, компилирует программу-исполнитель, присваивает бинарному файлу имя с автоматически вычисленным `flavor`, а также создаёт JSON-файл с описанием сборки, позволяющий впоследствии вспомнить, какой конфигурации соответствует данный хеш.

Для упаковки Python-интерпретатора и зависимостей в единый исполняемый файл мы использовали PyInstaller. Версия PyInstaller влияет на поведение получаемого бинарного файла, однако способ сборки не входит в описание среды исполнения Python. Поэтому такие детали не включаются в `flavor`, а их изменения можно учитывать через семантическую версию приложения BOINC. Сервер BOINC подхватывает такие обновления штатным образом

и не требует остановки.

Тем самым предложенный подход фактически разделяет задачу организации использования разных вариантов среды исполнения на три независимых уровня абстракции. На первом уровне находится способ сборки под конкретную архитектуру процессора и операционную систему, для логически одной среды эти способы могут быть устроены по-разному. На втором — описание окружения исполнения Python, задаваемое составом и версиями библиотек и интерпретатора. На третьем уровне BOINC-сервер оперирует компактным идентификатором `flavor`, выбирая для каждой платформы последнюю версию приложения. Такое разделение позволяет независимо развивать механизмы сборки и средства описания окружения исполнения, не требуя остановки, перезапуска или пересоздания проекта BOINC.

В результате предложенный подход обеспечивает переносимость вычислительных приложений между различными архитектурами и уменьшает зависимость этих приложений от программной конфигурации устройств добровольцев. Это снижает требования к узлам-участникам, расширяет круг устройств, которые могут быть подключены к системе STOILO, и делает решение удобным не только в эксплуатации, но и для дальнейшего развития.

7.4 Межверсионный транспорт Python-вызовов

Каждая задача состоит из по меньшей мере три вызываемых объекта: `func`, `init_valid_func` и `compare_valid_func`. Каждый из них в общем случае может захватывать внешний контекст и обладать внутренним состоянием. Кроме того, в состав задачи входят словарь Python-объектов, передаваемых в качестве аргументов, и Python-объект, представляющий возвращаемое значение.

Для сериализации всех сущностей, передаваемых от пользователя к серверу и далее на устройства добровольцев, используется библиотека `cloudpickle`. Её выбор обусловлен поддержкой сериализации широкого круга объектов, включая замыкания и объекты библиотек `numpy`, `pandas` и `pytorch`. Кроме того, `cloudpickle` используется и в других распределённых системах: например, Ray — распределённый фреймворк с future-ориентированной моделью вычислений — применяет эту библиотеку для сериализации `lambda`- и вложенных функций [18, 20], а `dask.distributed` — Python библиотека распределённых вычислений с интерфейсом `futures` — использует её для передачи пользовательских функций между компонентами системы [12, 13].

Однако использование `cloudpickle` для сериализации объектов, возвращаемых с устройств добровольцев на сервер, а затем передаваемых пользователям, недопустимо по со-

ображениям безопасности. Десериализация недоверенных данных с помощью `dill`, `pickle` или `cloudpickle` может приводить к выполнению произвольного кода. Поскольку возвращаемое значение формируется на стороне добровольца, его последующая десериализация на сервере или устройстве исследователя открывала бы возможность атаки удалённого выполнения кода. В условиях добровольной вычислительной сети, где отсутствует административный контроль над устройствами-исполнителями, такая уязвимость является критической и не может быть заложена в архитектуру системы.

Поэтому значения, возвращаемые функцией `func`, в целях безопасности должны быть JSON-сериализуемыми. Это ограничивает множество допустимых возвращаемых объектов комбинациями стандартных типов Python: словарей, списков, строк, чисел, булевых значений и `None`. На первый взгляд такое ограничение может показаться неудобным, однако в предлагаемой архитектуре оно является оправданным: возвращаемое значение должно быть десериализовано на сервере для последующей валидации, а значит, его представление не должно зависеть от типов из сторонних библиотек, доступных только на стороне исследователя и вычислительных узлов.

Важно, что это ограничение не мешает запускать вычисления со сложными пользовательскими функциями. Версии Python, `cloudpickle` и сторонних библиотек на вычислительном узле определяются выбранным вариантом исполнения, поэтому при передаче `func` и `kwargs` на устройства добровольцев можно обеспечить полную совместимость. На сервере ситуация иная: он использует собственные версии Python и `cloudpickle`. Поэтому принципиально важно, что на сервере десериализуются только сравнительно простые функции валидации, работающие со стандартным JSON-представлением результата. Различия версий не должны создавать существенных проблем для таких функций.

Таким образом, предложенная архитектура удачно сочетает выразительность Python с требованиями безопасности в добровольной вычислительной среде. Сложные пользовательские функции и их аргументы передаются на узлы-исполнители с помощью `cloudpickle` и выполняются в полностью совместимом варианте окружения исполнения, тогда как результат возвращается в JSON-совместимом виде и безопасно проверяется сервером. Благодаря этому система остаётся гибкой для исследователя, не требует изменения настроек сервера и избегает критического класса атак, связанных с десериализацией недоверенных объектов.

8 Машинное обучение в STOILO

В этом разделе демонстрируются возможности STOILO применительно к распределённому обучению моделей машинного обучения, прежде всего глубинного обучения. Для исследователя рабочий процесс остаётся привычным: разработка может вестись в интерактивной среде Jupyter Notebook с использованием стандартных библиотек Python, а основные абстракции машинного обучения — модель, функция потерь, оптимизатор и датасет — остаются явно доступными пользователю. В типовых сценариях пользователь не обязан напрямую работать с интерфейсом асинхронных задач: мы реализовали несколько обёрток, которые отвечают за разбиение данных, формирование задач, их отправку на узлы-исполнители и сбор результатов.

При этом STOILO не ограничивает исследователя готовыми обёртками. Если требуется реализовать нестандартную стратегию обучения, интерфейс асинхронных задач позволяет описывать вычисления обычными Python-функциями с произвольными библиотечными зависимостями, задавать связи между отдельными задачами и собирать из них нужную схему выполнения. Такой подход оставляет пользователю полный контроль над алгоритмом обучения: параметры модели, вычисления на фрагментах данных, агрегация результатов и шаг оптимизации выражаются явно средствами привычных библиотек, без необходимости подстраиваться под заранее заданный узкий шаблон распределённой системы.

В наиболее общем виде машинное обучение можно рассматривать как итеративный процесс поиска параметров модели, обеспечивающих требуемое качество. В начале обучения задаются архитектура модели, стартовые значения её параметров и состояние оптимизатора. Затем модель с текущими параметрами применяется к обучающим данным, а результаты вычислений интерпретируются выбранным алгоритмом оптимизации. На основании этой информации формируется новое состояние модели, после чего процесс повторяется до выполнения критерия остановки.

Такой процесс естественно распараллеливается по данным. Текущее состояние модели передаётся на узлы добровольцев в составе независимых задач, каждая из которых выполняет вычисления на своём фрагменте обучающих данных. Возвращённые результаты агрегируются и используются для построения новых параметров модели. В классической схеме обучения нейронных сетей такими результатами являются градиенты, полученные при обратном распространении ошибки, а обновление выполняется шагом градиентного спуска. Однако архитектура STOILO не фиксирует ни устройство модели, ни способ обновления её параметров: та же схема применима и к другим методам оптимизации, включая даже

8.1 Синхронный градиентный спуск

Наиболее естественный способ распределить обучение по данным состоит в том, чтобы сохранить привычную схему градиентного спуска, но вынести вычисление градиентов на отдельные узлы. На каждом шаге обучения текущее состояние модели фиксируется и используется для формирования набора независимых задач. Каждая задача получает собственный фрагмент выбранной части обучающей выборки, выполняет прямой проход, вычисляет значение функции потерь и производит обратное распространение ошибки. Возвращаемым результатом задачи являются градиенты функции потерь по параметрам модели и значение функции потерь на соответствующем фрагменте данных. После завершения всех задач градиенты усредняются, и оптимизатор выполняет один шаг обновления параметров.

Важной особенностью такой схемы является то, что один шаг обучения не обязан соответствовать обработке всего обучающего датасета. В терминах PyTorch это может быть реализовано через выделение подмножества данных с помощью `torch.utils.data.Subset` и построение нового `DataLoader` на каждом шаге обучения. Таким образом, эпоха состоит из нескольких агрегированных шагов: на каждом шаге выбирается новая часть обучающей выборки, разбивается на мини-батчи, после чего по результатам их обработки выполняется одно обновление параметров модели. Эта схема ближе к классическому мини-батч обучению, чем к полному батчёмому градиентному спуску, поскольку оптимизатор выполняет несколько обновлений в пределах одной эпохи.

Для исследователя данная схема выглядит почти как обычное обучение PyTorch модели. Пользователь по-прежнему работает с моделью, датасетом, функцией потерь и оптимизатором, однако явное вычисление градиентов на отдельных батчах передаётся объекту `SyncDataParallelTrainer`. Он берёт на себя формирование задач, их запуск на узлах-исполнителях, сбор результатов и усреднение градиентов.

```

1 import torch
2 from torch.utils.data import DataLoader, Subset
3
4 from stoilo.ddl import SyncDataParallelTrainer
5
6 trainer = SyncDataParallelTrainer(
7     conn=stoilo_connection,
8     model=cifar10_resnet18,
9     loss_function_factory=torch.nn.CrossEntropyLoss,
10    loss_function_parameters={},
11    optimizer_factory=torch.optim.Adam,
12    optimizer_parameters={"lr": 0.001},
13 )

```

```

14
15 subset_size = 4096
16 batch_size = 128
17 steps_per_epoch = 12
18
19 loss_history = []
20 for epoch_index in range(num_epochs):
21     for step_index in range(steps_per_epoch):
22         subset_indices = torch.randperm(len(cifar10_train_dataset))[:subset_size]
23         training_subset = Subset(cifar10_train_dataset, subset_indices)
24
25         step_loader = DataLoader(
26             training_subset,
27             batch_size=batch_size,
28             shuffle=True,
29             drop_last=False,
30         )
31
32         step_metrics = await trainer.train_step(
33             epoch_index=epoch_index,
34             step_index=step_index,
35             train_loader=step_loader,
36         )
37
38         loss_history.append(step_metrics.average_loss)
39
40 trained_model = trainer.get_model()

```

Листинг 6: Синхронное распределённое обучение модели PyTorch с выбором подмножества данных на каждом шаге.

В листинге 6 идентификатор `cifar10_train_dataset` соответствует части датасета, используемой для обучения. На каждом шаге из неё выбирается новое подмножество `training_subset`, для которого создаётся отдельный загрузчик данных `step_loader`. Именно этот загрузчик передаётся в метод `trainer.train_step`, внутри которого данные посредством загрузчика разбиваются на независимые фрагменты и отправляются на узлы-исполнители.

Каждый узел вычисляет градиенты функции потерь по параметрам модели на своём фрагменте данных. После получения всех результатов `SyncDataParallelTrainer` усредняет градиенты с учётом размера соответствующих фрагментов и передаёт итоговый градиент оптимизатору. Оптимизатор выполняет шаг обновления параметров модели. Таким образом, один шаг оптимизации строится не по одному мини-батчу, а по совокупности нескольких независимо обработанных батчей. Первоначально исследовался вариант полного градиентного спуска по всей обучающей выборке, однако его результаты оказались неудовлетворительными вследствие слишком низкой частоты обновления параметров модели.

Достоинство синхронной схемы состоит в её простоте и близости к классическому обучению с усреднением градиентов. Каждый шаг обновления параметров выполняется по результатам обработки достаточно большой части обучающих данных, поэтому поведение

алгоритма остаётся понятным и хорошо воспроизводимым. Кроме того, такая схема удобна для проверки корректности распределённой реализации: все результаты текущего шага известны до обновления модели, а значит, легко сравнивать распределённый вариант с локальным обучением.

Однако в синхронную схему заложено существенное ограничение: шаг обновления выполняется только после завершения всех задач текущей итерации. Следовательно, время итерации определяется самым медленным узлом распределённой системы. В добровольной вычислительной сети, где устройства различаются по производительности, доступности и сетевой задержке, это может снижать эффективность распараллеливания.

8.2 Частично синхронный градиентный спуск

Полностью синхронная схема предполагает ожидание завершения всех задач текущего шага перед переходом к следующему. В добровольной вычислительной сети это делает время шага зависимым от наиболее медленных участников. Для уменьшения данного влияния разумно использовать частично синхронный вариант, в котором шаг оптимизатора выполняется после получения достаточного числа результатов, а остальные вычисления считаются запаздывающими и отбрасываются.

Такая схема отличается от синхронной только правилом завершения шага. Разбиение данных, создание `DataLoader`, отправка задач и усреднение градиентов остаются теми же. Однако агрегируются не все вычисленные градиенты, а только те, которые были получены до момента принятия решения об обновлении модели. Запаздывающие результаты, поступившие после этого момента, соответствуют предыдущим состояниям модели и в рамках нашего алгоритма в агрегации не участвуют. При этом возможны и другие, стохастические схемы, учитывающие такие результаты.

```
1 from stoilo.ddl import StaleSyncDataParallelTrainer
2
3 trainer = StaleSyncDataParallelTrainer(
4     ...
5     min_completed_fraction=0.875,
6     discard_late_results=True,
7 )
```

Листинг 7: Параметры частично синхронного распределённого обучения.

Листинг 7 показывает только отличие от синхронного варианта. Параметр `min_completed_fraction` задаёт минимальную долю результатов, достаточную для выполнения шага оптимизатора. Например, если выбранное подмножество данных разбивается на

32 задачи, то значение `min_completed_fraction=0.875` требует дождаться 28 результатов и позволяет отбросить до четырёх наиболее медленных. Параметр `discard_late_results` указывает, что результаты, пришедшие после обновления модели, не применяются к новому состоянию параметров.

Частично синхронная схема вносит дополнительную стохастичность, поскольку разные шаги могут использовать разные подмножества завершившихся задач. При этом она сохраняет основную идею синхронного усреднения градиентов, но позволяет переходить к следующему шагу, а при необходимости и к следующей эпохе, не дожидаясь самых медленных узлов. В условиях сети добровольных вычислений это очень ценное свойство.

9 Тестирование и эксплуатация

9.1 Средства развёртывания STOILO

Серверная часть системы снабжена продуманным механизмом развёртывания, состоящем из `Dockerfile`, `entrypoint.sh` и ряда конфигурационных файлов и вспомогательных скриптов. Совокупность этих компонентов позволяет одной командой собрать Docker-образ, который при запуске самостоятельно проинициализирует и запустит сервер. Логика Docker-контейнера включает ожидание доступности базы данных, установку соединения с БД, создание необходимой файловой и табличной структуры BOINC проекта, настройку прав доступа, регистрацию версий приложений, запуск веб-сайта, административной панели и всех необходимых демонов, включая процесс `supervisord` для выполнения периодических служебных задач и автоматического перезапуска демонов, завершившихся с ошибкой.

Для целей тестирования, демонстрации и проведения экспериментов мы разработали Docker-контейнер, имитирующий добровольный вычислительный узел. Файлы образа находятся в директории `volunteer`. Контейнер запускает клиент BOINC и автоматически выполняет ряд шагов для подключения к проекту, включая ожидание доступности сервера и создание нового уникального аккаунта. Контейнер устойчив как к собственным перезапускам — в этом случае он повторно подключается к уже созданному аккаунту, — так и к сбоям сервера и разрывам сетевого соединения.

В корневой директории репозитория находится `docker-compose.yaml`, позволяющий поднять систему локально для целей разработки и тестирования. Этот способ запуска активно использовался на ранних этапах, пока механизмы развёртывания системы не были полностью отлажены.

9.2 Развёртывание сервера в облаке

Для приближения к сценариям реального применения, серверная часть была развёрнута в Яндекс Облаке. Сервер STOILO и база данных MariaDB были размещены в отдельных контейнерных виртуальных машинах с умеренными вычислительными ресурсами: 2 vCPU, 4 ГБ оперативной памяти и 60 ГБ дискового пространства. Для обеспечения связи с инсталляцией было приобретено доменное имя `stoilo.sermir-tech.ru`. С помощью нативного облачного сервиса балансировки нагрузки был выпущен TLS-сертификат и настроено защищённое подключение по HTTPS.

В рамках разработческой инсталляции порт соединения Python-библиотеки с сервером не открывался для доступа из Интернета, а был доступен через SSH-туннель; вопросы контроля доступа в используемой на практике инсталляции рассмотрены в разделе 10.

9.3 Разнородные физические узлы

Для проверки работоспособности системы в условиях добровольной вычислительной сети, состоящей из разнообразных устройств, скрытых за NAT, был проведён эксперимент, в котором задачи из `python_lib/examples/low_level.ipynb` выполнялись на трёх компьютерах с различными архитектурами процессора и операционными системами: Linux x86_64 на процессоре AMD, Windows x86_64 на процессоре Intel и macOS Arm64 на процессоре Apple Silicon. Использование разнородных устройств позволило проверить ключевое свойство STOILO: способность одной серверной инсталляции выдавать задачи BOINC-клиентам разных платформ и запускать на них соответствующие сборки исполнения.

Фотографии развёртывания системы на трёх физических устройствах в рамках эксперимента приведены в приложении A.

Простые примеры из `low_level.ipynb` проверили базовый пользовательский сценарий STOILO: создание асинхронных задач, передачу Python-функций и аргументов, выполнение вычислений на добровольных узлах и возврат результатов пользователю. В рамках этого сценария были задействованы ключевые внутренние механизмы системы: сериализация пользовательской функции и входных данных, формирование заданий для BOINC, доставка исполняемого окружения на клиентские устройства, запуск вычислений через программное обеспечение добровольца, передача результата обратно на сервер, его последующая проверка валидаторами и обработка различных ошибок.

Таким образом, эксперимент подтвердил работоспособность основных компонентов системы и показал, что они корректно взаимодействуют в полном цикле выполнения поль-

зовательских задач.

9.4 Прерываемые контейнерные узлы

В следующем эксперименте проверялась работа системы в конфигурации с несколькими независимыми вычислительными узлами, размещёнными в облачном сервисе. Для этого были запущены пять прерываемых виртуальных машин с конфигурацией 2 vCPU, 4 ГБ оперативной памяти и 20 ГБ дискового пространства. На каждой из них работал подготовленный контейнер-имитатор добровольца: он запускал BOINC-клиент, подключался к проекту STOILO и выполнял получаемые задачи так же, как обычный добровольный узел.

Вытесняемые облачные узлы хорошо согласуются с моделью добровольных вычислений, поскольку такая модель изначально учитывает возможность внезапного исчезновения исполнителей. В таком сценарии облачный провайдер может остановить виртуальную машину без предварительного согласования с пользователем; BOINC-инфраструктура при этом обнаруживает факт невыполнения задачи и при необходимости назначает её другому исполнителю. Использование вытесняемых контейнеров также перспективно благодаря их меньшей стоимости и возможности быстро разворачивать вычислительные кластеры большого размера.

В данной полностью облачной инсталляции были запущены как простые примеры из `low_level.ipynb`, так и примеры распределённого глубинного обучения. Этот запуск дополнил предыдущую проверку и позволил убедиться, что система сохраняет работоспособность при использовании многих независимых облачных исполнителей.

9.5 Проверка обёрток для машинного обучения

Отдельно была проверена работа обёрток для распределённого глубинного обучения, реализованных поверх интерфейса Python-функций. Цель проверки заключалась в том, чтобы доказать, что предложенная схема позволяет выполнить полный цикл обучения PyTorch-модели инструментами PyTorch: сформировать задачи, вычислить градиенты на фрагментах данных, агрегировать результаты и выполнить обновление параметров.

В качестве тестовой задачи была выбрана классификация изображений CIFAR-10 с помощью модели ResNet18 без предобучения. На каждом шаге из обучающего датасета выбиралось подмножество объектов, после чего оно разбивалось на несколько непересекающихся батчей. Для каждого батча вычислялись значение функции потерь и градиенты по параметрам модели. Затем градиенты агрегировались, и по результатам агрегации выполнялся шаг

оптимизации.

Были проверены два варианта обучения. В первом варианте использовалась полностью синхронная схема: обновление модели выполнялось после получения результатов по всем батчам текущего шага. Во втором варианте применялась частично синхронная схема, в которой небольшая доля наиболее поздних результатов могла быть отброшена. Оба варианта показали устойчивое уменьшение функции потерь и рост качества классификации на валидационной выборке.

На рисунке 9.1 приведена динамика функции потерь в начале обучения. В обоих вариантах функция потерь быстро убывает на первых эпохах, после чего снижение становится более плавным. Такое поведение соответствует ожидаемой динамике обучения нейронной сети. Напомним, что цель состояла не в получении модели с высокими метриками качества, а в проверке жизнеспособности концепции STOILO. В данном случае важен сам факт того, что процесс обучения выполняется, а качество модели улучшается.

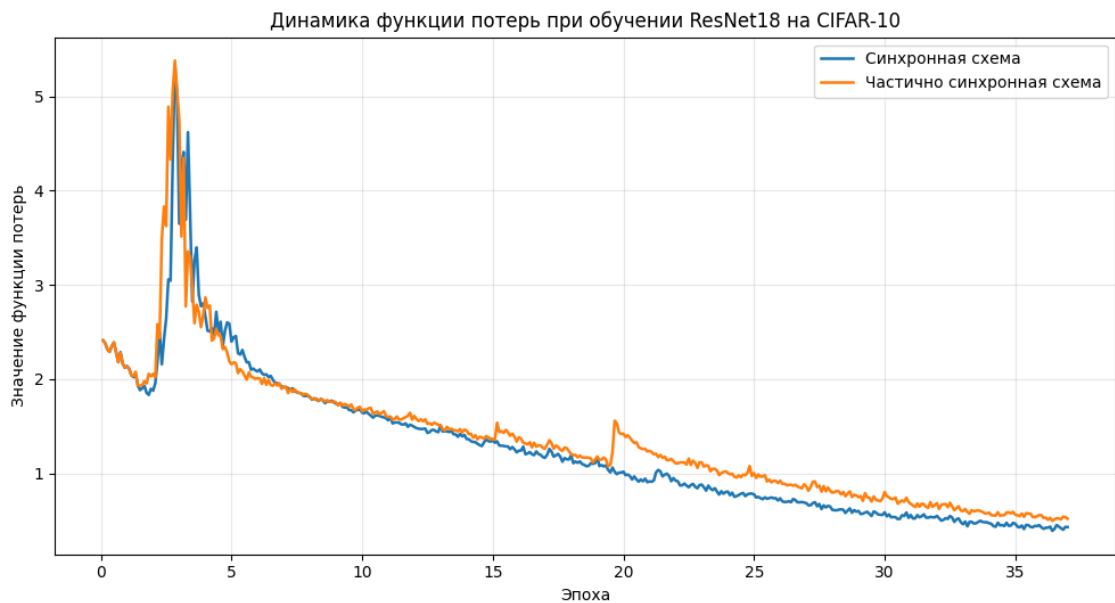


Рис. 9.1: Динамика функции потерь при обучении ResNet18 на CIFAR-10 с использованием обёрток распределённого обучения.

По итогам эксперимента точность классификации на валидационной выборке достигала уровня около 80% для синхронной схемы и 78% для частично синхронной. Близкие значения были получены для обеих схем агрегации, что показывает корректность общей логики распределённого обучения и возможность использования обёрток для практических задач машинного обучения. При этом частично синхронный вариант сохранял качество, сопоставимое с полностью синхронным, но допускал переход к следующему шагу без ожидания наиболее медленных результатов.

10 Безопасность

STOILO проектировалась для гибкого выполнения пользовательских Python-функций в добровольной вычислительной сети. Данная концепция естественным образом порождает серьёзные вопросы безопасности: исследователь получает возможность запускать на устройствах добровольцев произвольный код, добровольные узлы возвращают сериализованные и потенциально неверные результаты системе, а серверная инсталляция должна быть защищена от несанкционированного планирования задач. В этом разделе описываются границы доверия, принятые в архитектуре STOILO, и механизмы, снижающие основные риски.

10.1 Безопасность устройств добровольцев

В классической модели BOINC доброволец подключается к проекту и исполняет код, предоставляемый этим проектом. BOINC имеет встроенные механизмы контроля подлинности и целостности файлов: версии приложений подписываются приватным ключом проекта, клиент BOINC запускает только программы с корректной подписью, а передача файлов выполняется по HTTPS [9, 7]. Доброволец может быть уверен, что исполняемый файл был выпущен проектом, а его содержимое не изменилось при передаче по сети.

Проверка подлинности файлов, однако, не гарантирует безопасность самого кода. В STOILO этот вопрос особенно важен, поскольку система позволяет исследователям выполнять на узлах добровольцев произвольные Python функции. В базовой модели предполагается, что доброволец доверяет проекту, к которому подключается. В случае STOILO это доверие распространяется и на исследователей, которым разрешено планирование задач.

Основным способом снижения риска для добровольца остаётся изоляция BOINC клиента и выполняемых приложений. Например, запуск в виртуальной машине, контейнере или иной среде с ограниченными правами. Такой подход соответствует общей практике BOINC, где виртуальная машина рассматривается как наиболее надёжно изолированная среда исполнения для недоверенного кода [8].

Более тонкие ограничения пользовательского Python-кода возможны, но нуждаются в отдельном проектировании и не устраняют необходимость доверять проекту. STOILO не решает эту задачу полностью на уровне фреймворка: безопасность устройств добровольцев должна обеспечиваться прежде всего внешней изоляцией и контролем доступа исследователей к планированию задач.

10.2 Безопасность сервера и исследователя

Сервер и исследователь, в свою очередь, не обязаны доверять устройствам добровольцев. Узел-исполнитель может вернуть неверный результат вследствие ошибки, технических проблем или злонамеренного поведения. Кроме того, обработка результатов вычислений не должна допускать выполнения произвольного кода.

Ключевое архитектурное решение STOILO состоит в том, что библиотека `cloudpickle` используется только для передачи данных от исследователя к узлам-исполнителям и не применяется для обработки результатов, возвращаемых добровольцами. Это принципиально важно: десериализация данных средствами `pickle`, `dill` или `cloudpickle` может приводить к выполнению произвольного кода. Официальная документация Python прямо предупреждает, что модуль `pickle` не предназначен для работы с недоверенными данными [19].

Поэтому значения, возвращаемые функцией `func`, должны быть JSON сериализуемыми. Это ограничение снижает выразительность возвращаемых значений по сравнению с произвольными Python-объектами, но устраняет критический класс уязвимостей, связанных с небезопасной десериализацией.

Для проверки корректности результатов STOILO использует механизмы валидации, согласованные с моделью BOINC. BOINC рассчитан на возможность получения ошибочных или намеренно некорректных результатов и предоставляет механизмы их проверки, основанные на вычислительной репликации и использовании пользовательских валидаторов [9]. В STOILO эта идея переносится на Python-интерфейс: исследователь может задавать собственные функции проверки результата. Поскольку валидаторы работают со стандартным JSON-представлением, они не требуют десериализации произвольных объектов, присланных добровольцем.

При этом важно различать недоверенных добровольцев и доверенных исследователей. STOILO защищает сервер и исследователя от потенциально некорректных или вредных результатов, полученных от добровольцев, однако не рассматривает авторизованного исследователя как источник угрозы для инсталляции.

10.3 Контроль доступа к планированию задач

Отдельной задачей является защита инсталляции STOILO от планирования задач неавторизованными пользователями. В текущую реализацию STOILO полноценная система аутентификации и авторизации не входит. Это не является недостатком архитектуры: самостоятельная реализация таких механизмов без специализированной экспертизы создаёт

неоправданные риски, тогда как на практике для защиты служебных интерфейсов обычно используют проверенные инфраструктурные решения.

Доступ к интерфейсу планирования задач может быть ограничен на сетевом или транспортном уровне: через подключение к виртуальной приватной сети организации (корпоративный VPN), SSH-туннелирование до сервера или обратный прокси с аутентификацией. Эти способы широко применяются на практике для защиты служебных интерфейсов.

В исследовательской инсталляции, поднимаемой в данной работе, доступ к служебному интерфейсу был организован через SSH-туннель до внутреннего порта сервера. Такое решение не требует развёртывания отдельной инфраструктуры доступа и хорошо подходит для разработки.

Таким образом, безопасность STOILO строится на явном разделении границ доверия. Доброволец доверяет проекту и при необходимости использует изолированную среду выполнения; сервер не доверяет добровольцам, принимает только безопасно десериализуемые JSON-результаты и выполняет их валидацию; исследователи находятся в зоне административной ответственности оператора инсталляции, который ограничивает доступ к интерфейсу планирования задач стандартными инфраструктурными средствами. Такое разделение делает архитектуру практически применимой и явно определяет ответственность каждой стороны.

11 Направления дальнейшей работы

11.1 Развитие обёрток для машинного обучения

STOILO создавалась прежде всего как система для распределённого глубинного обучения. Чтобы быть полезной на практике, она должна позволять исследователям работать с привычными библиотеками, обучать произвольные модели и самостоятельно выбирать способ организации вычислений. Для этого потребовалось решить более общую инфраструктурную задачу: предоставить к системе на базе BOINC асинхронный интерфейс планирования Python-функций с настраиваемым окружением исполнения. Именно этой задаче уделено основное внимание в данной работе.

В результате была создана инфраструктура, позволяющая передавать пользовательские функции, задавать окружения исполнения, запускать вычисления на разнородных узлах и получать результаты через Python-библиотеку. Поверх неё были реализованы обёртки, демонстрирующие обучение моделей PyTorch с использованием параллелизма по данным.

Они показывают, что исследователь может работать с привычными сущностями машинного обучения привычными способами.

Вместе с тем реализованные обёртки являются только первым шагом. Для полноценного применения STOILO в машинном обучении необходимо развивать более высокоуровневые интерфейсы и исследовать разные стратегии распределённого обучения: способы агрегации результатов, режимы выдачи задач, обработку запаздывающих обновлений, подбор гиперпараметров и критерии остановки. Эти вопросы требуют отдельной проработки и могут стать предметом дальнейших исследований.

11.2 Избыточная репликация вычислений

Поскольку в среде добровольных вычислений необходимо быть готовым к недобросовестному поведению и попыткам саботажа со стороны волонтёров, каждая задача вычисляется по меньшей мере два раза, и считается выполненной только после того как по меньшей мере два добровольца вернули эквивалентные результаты. Такой подход хорошо согласуется с моделью BOINC, однако кратно увеличивает объём вычислений.

Для задач машинного обучения эта цена особенно заметна. Если результатами отдельных задач являются, например, градиенты или локально обновлённые параметры модели, полная репликация каждого вычисления напрямую замедляет обучение. Кроме того, сама по себе репликация не является абсолютной защитой: при наличии достаточного числа контролируемых аккаунтов злоумышленник теоретически может вернуть одинаковый неправильный результат несколько раз и тем самым обойти простую схему сравнения ответов.

Перспективным направлением дальнейшей работы является более экономная схема валидации. Например, можно реплицировать не все вычисления, а только некоторую их долю, использовать доверенные узлы для выборочной перепроверки результатов или назначать дополнительную проверку исполнителям с подозрительной историей.

В задачах машинного обучения умеренное число ошибочных обновлений параметров или градиентов обычно не приводит к срыву оптимизации, поэтому такая схема может существенно уменьшить накладные расходы, сохранив практическую устойчивость системы. Разработка такой схемы потребуют серьёзных изменений в работе программного обеспечения BOINC, но позволит значительно ускорить обучение моделей.

12 Заключение

В рамках данной работы были проанализированы существующие подходы к организации распределённого глубинного обучения, а также изучены системы и проекты в области добровольных вычислений. На основе собранных знаний и выявленных ограничений существующих решений разработана новая система, позволяющая запускать произвольные Python-задачи на распределённых вычислительных узлах в добровольной среде. Система обладает удобным интерфейсом в виде Python-библиотеки, а её инсталляция в облачном сервисе из Docker-образа не представляет труда.

До настоящего времени задача гибкого исполнения произвольного кода в вычислительной системе на основе BOINC не имела удовлетворительного решения. В рамках данной работы предложена и реализована оригинальная система, исходный код которой открыт и доступен для модификаций и конечного использования.

Система позволяет запускать Python функции в полностью настраиваемом окружении исполнения: исследователь может выбрать версию интерпретатора и требуемый набор сторонних библиотек и их версий. Предложенная концепция совмещения гибко настраиваемых окружений исполнения с моделью версий приложений BOINC настолько элегантна, что планирование задач, описание среды исполнения и сборка среды для конкретных платформ находятся на разных уровнях абстракции и могут развиваться независимо друг от друга. Для каждого из этих уровней в работе предложены собственные автоматизированные решения.

Отдельно от серверной части и узлов добровольцев развивается Python-библиотека. Интерфейс асинхронных функций позволяет описывать сложные зависимости между задачами и проводить вычисления параллельно. При этом сама парадигма таких вычислений не нова, поэтому пользователи без труда смогут адаптировать логику своих вычислений к предлагаемой модели.

Разработанная система предоставляет универсальный механизм удалённого выполнения Python-кода, что делает её применимой для широкого круга задач распределённых вычислений. Для демонстрации работоспособности концепции были реализованы обёртки для параллельного по данным обучения модели: с синхронным и асинхронным стохастическим градиентным спуском.

Хотя исполнение произвольного кода на узлах добровольцев сопряжено с определёнными угрозами безопасности, высокая адаптивность созданной системы делает её перспективной для дальнейшей разработки, совершенствования и применения в решении задач реальных исследовательских проектов.

А Физические устройства тестовой инсталляции

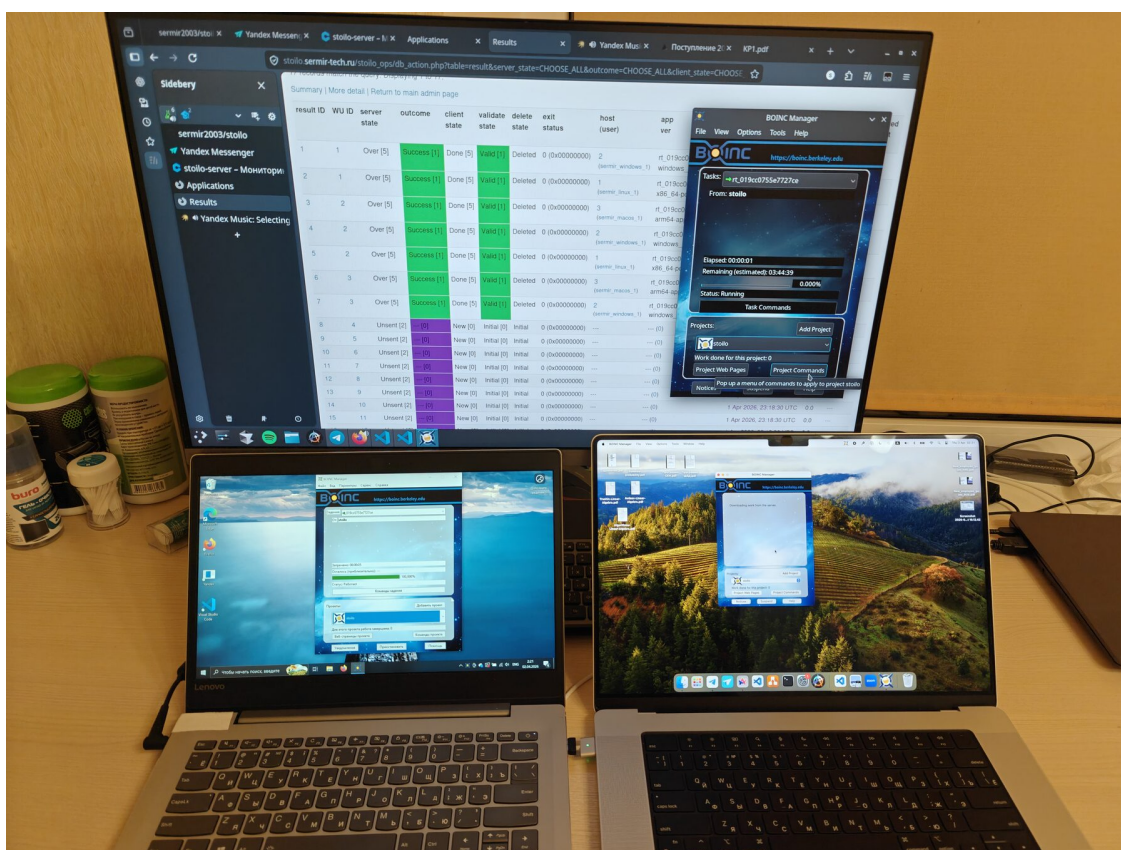


Рис. А.1: Тестовая инсталляция STOILO на трёх физических устройствах: Linux x86_64, Windows x86_64 и macOS Arm64.

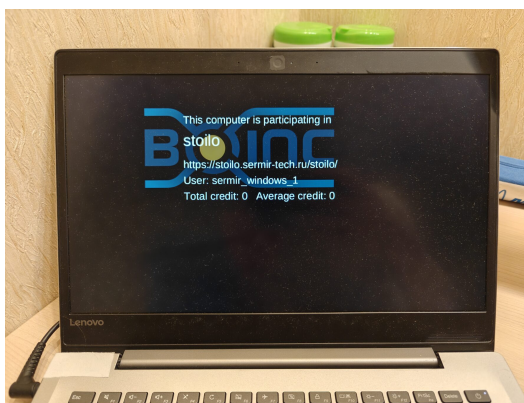


Рис. А.2: Экранная заставка ноутбука, участвующего в проекте STOILO.

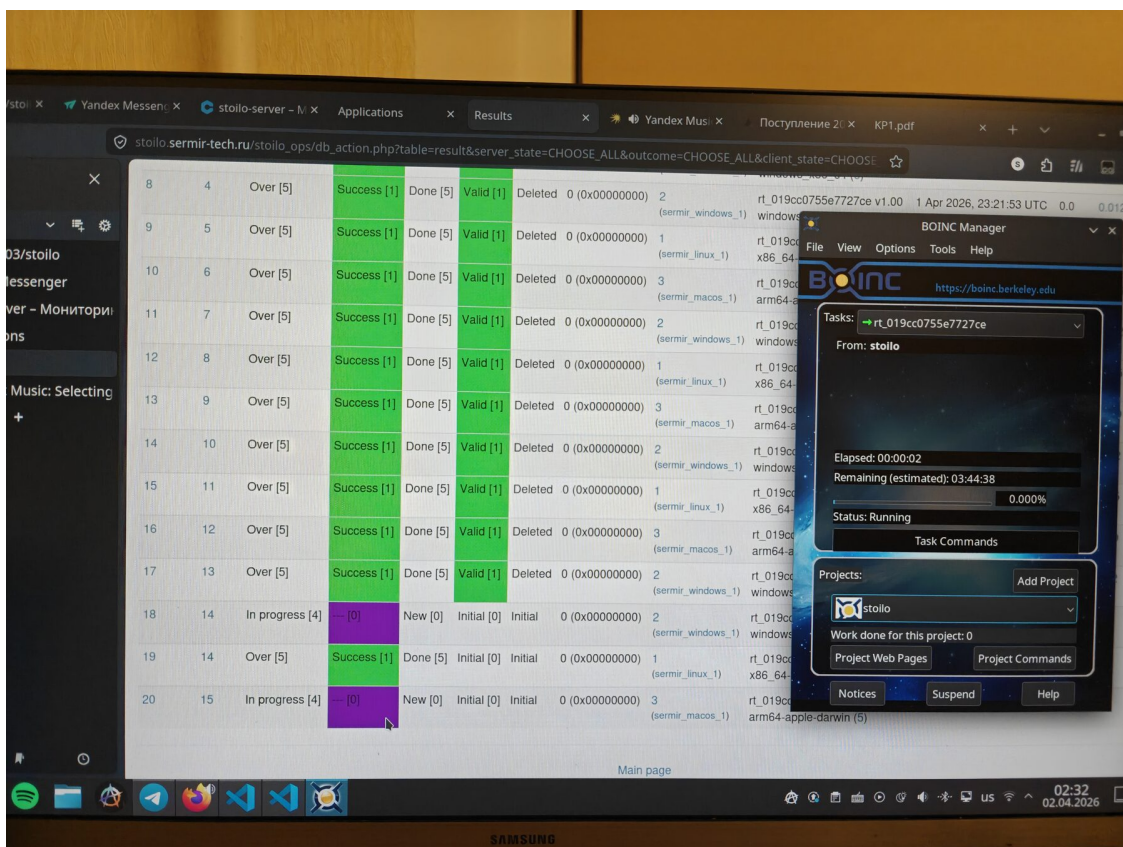


Рис. А.3: Панель администратора BOINC, обратите внимание на URL.

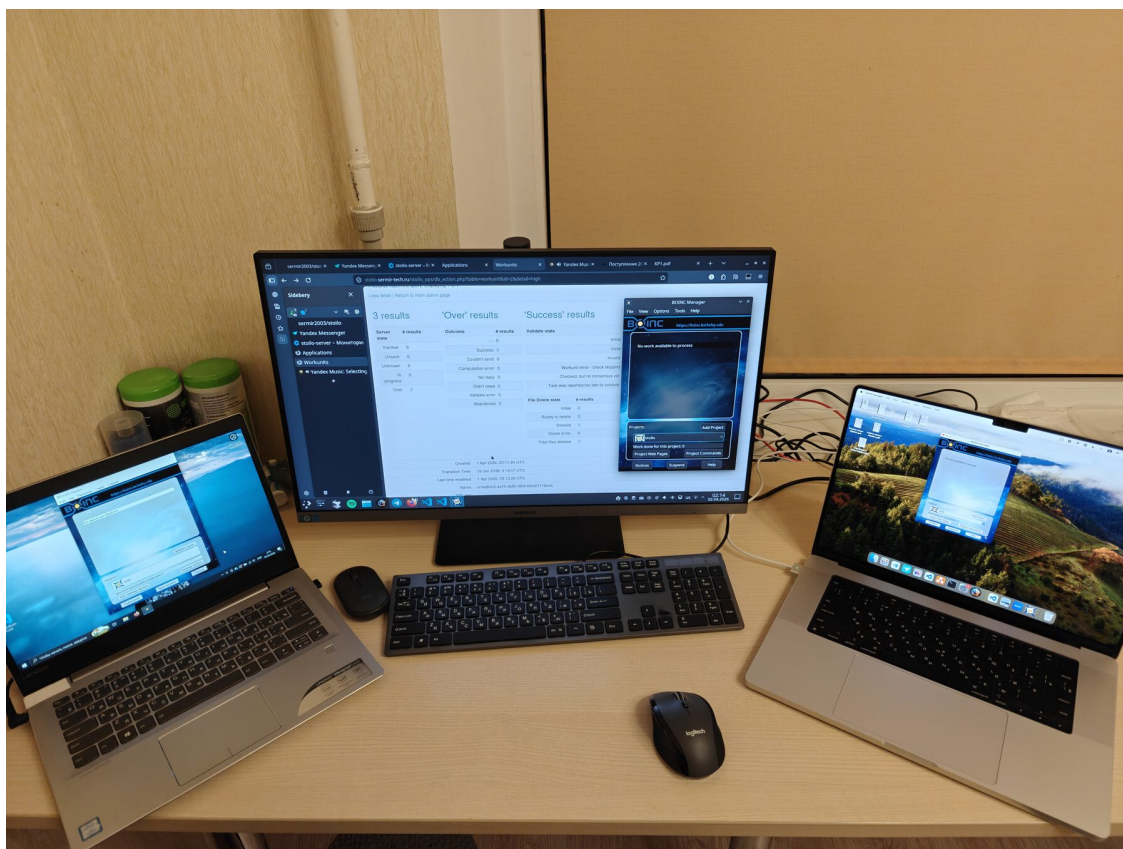


Рис. А.4: Три очень разных компьютера, участвующих в STOILO.

Список литературы

- [1] Saúl Alonso-Monsalve, Félix García-Carballeira и Alejandro Calderón. “A New Volunteer Computing Model for Data-Intensive Applications”. B: *Concurrency and Computation: Practice and Experience* 29.24 (2017), e4198. DOI: [10.1002/cpe.4198](https://doi.org/10.1002/cpe.4198).
- [2] D. P. Anderson. “BOINC: A System for Public-Resource Computing and Storage”. B: *Proceedings of the Fifth IEEE/ACM International Workshop on Grid Computing (GRID)*. 2004, с. 4—10. DOI: [10.1109/GRID.2004.14](https://doi.org/10.1109/GRID.2004.14).
- [3] David Anderson. “BOINC: A Platform for Volunteer Computing”. B: *Journal of Grid Computing* 18.1 (2020), с. 99—122. DOI: [10.1007/s10723-020-09509-9](https://doi.org/10.1007/s10723-020-09509-9).
- [4] David P. Anderson, Jeff Cobb, Eric Korpela, Matt Lebofsky и Dan Werthimer. “SETI@home: An Experiment in Public-Resource Computing”. B: *Communications of the ACM* 45.11 (2002), с. 56—61. DOI: [10.1145/581495.581497](https://doi.org/10.1145/581495.581497).
- [5] Medha Atre, Birendra Jha и Ashwini Rao. “Distributed Deep Learning Using Volunteer Computing-Like Paradigm”. B: *Proceedings of the 2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2021, с. 933—942. DOI: [10.1109/IPDPSW52791.2021.00144](https://doi.org/10.1109/IPDPSW52791.2021.00144).
- [6] Tal Ben-Nun и Torsten Hoefler. “Demystifying Parallel and Distributed Deep Learning: An In-depth Concurrency Analysis”. B: *ACM Computing Surveys* 52.4 (апр. 2019), 65:1—65:43. ISSN: 0360-0300. DOI: [10.1145/3320060](https://doi.org/10.1145/3320060). URL: <https://doi.org/10.1145/3320060>.
- [7] BOINC Project. *Code Signing*. <https://github.com/BOINC/boinc/wiki/Code-signing>. Accessed: 2026-05-10.
- [8] BOINC Project. *Running Apps in VirtualBox Virtual Machines*. <https://github.com/BOINC/boinc/wiki/VboxApps>. Accessed: 2026-05-10.
- [9] BOINC Project. *Security in Volunteer Computing*. <https://github.com/BOINC/boinc/wiki/Security-in-volunteer-computing>. Accessed: 2026-05-10.
- [10] James Clemens. “MLDS: A Dataset for Weight-Space Analysis of Neural Networks”. B: *arXiv preprint arXiv:2104.10555* (2021). URL: <https://arxiv.org/abs/2104.10555>.
- [11] Fernando Costa, Luis Silva, Gilles Fedak и Ian Kelley. “Optimizing the Data Distribution Layer of BOINC with BitTorrent”. B: *Proceedings of the 2008 IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*. 2008, с. 1—8. DOI: [10.1109/IPDPS.2008.4536446](https://doi.org/10.1109/IPDPS.2008.4536446).

- [12] Dask Developers. *Dask.distributed — Dask.distributed Documentation*. <https://distributed.dask.org/>. Accessed: 2026-05-08.
- [13] Dask Developers. *Protocol — Dask.distributed Documentation*. <https://distributed.dask.org/en/latest/protocol.html>. Accessed: 2026-05-08.
- [14] Travis Desell. “Large Scale Evolution of Convolutional Neural Networks Using Volunteer Computing”. B: *arXiv preprint arXiv:1703.05422* (2017). URL: <https://arxiv.org/abs/1703.05422>.
- [15] Ian Foster, Carl Kesselman и Steven Tuecke. “The Anatomy of the Grid: Enabling Scalable Virtual Organizations”. B: *International Journal of High Performance Computing Applications* 15.3 (2001), с. 200—222. DOI: [10.1177/109434200101500302](https://doi.org/10.1177/109434200101500302).
- [16] Kaiming He, Xiangyu Zhang, Shaoqing Ren и Jian Sun. *Deep Residual Learning for Image Recognition*. 2015. arXiv: [1512.03385 \[cs.CV\]](https://arxiv.org/abs/1512.03385). URL: <https://arxiv.org/abs/1512.03385>.
- [17] Teshome M. Mengistu и Dandan Che. “Survey and Taxonomy of Volunteer Computing”. B: *ACM Computing Surveys* 52.3 (2019), 59:1—59:35. DOI: [10.1145/3320073](https://doi.org/10.1145/3320073).
- [18] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan и Ion Stoica. “Ray: A Distributed Framework for Emerging AI Applications”. B: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, окт. 2018, с. 561—577. ISBN: 978-1-939133-08-3. URL: <https://www.usenix.org/conference/osdi18/presentation/moritz>.
- [19] Python Software Foundation. *pickle — Python Object Serialization*. <https://docs.python.org/3/library/pickle.html>. Accessed: 2026-05-10.
- [20] Ray Project. *Serialization — Ray Documentation*. <https://docs.ray.io/en/latest/ray-core/objects/serialization.html>. Accessed: 2026-05-08.
- [21] Nils Schlitter и Jörg Lässig. “Distributed Data Analytics using RapidMiner and BOINC”. B: *Proceedings of the 4th RapidMiner Community Meeting and Conference (RCOMM)*. Porto, Portugal, 2013, с. 81—95.
- [22] Cristiano Tapparello, Carmelo Funai, Saif Hijazi и Wendi Heinzelman. “Volunteer Computing on Mobile Devices: State of the Art and Future Research Directions”. B: *Mobile Computing and Wireless Networks: Concepts, Methodologies, Tools, and Applications*. IGI Global, 2015, с. 125—153. DOI: [10.4018/978-1-4666-8662-5.ch005](https://doi.org/10.4018/978-1-4666-8662-5.ch005).

- [23] Joost Verbraeken, Matthijs Wolting, Jonathan Katzy, Jeroen Kloppenburg, Tim Verbelen и Jan S. Rellermeyer. “A Survey on Distributed Machine Learning”. B: *ACM Computing Surveys* 53.2 (март 2020), 30:1—30:33. ISSN: 0360-0300. DOI: [10 . 1145 / 3377454](https://doi.org/10.1145/3377454). URL: <https://doi.org/10.1145/3377454>.
- [24] Wikipedia contributors. *Rosetta@home*. <https://en.wikipedia.org/wiki/Rosetta@home>. Accessed: 2025-04-26. 2024.
- [25] Michael I. Zimmerman, J. R. Porter, Michael D. Ward, Siddharth Singh, Neha Vithani, Amit Meller, Uma Mallimadugula, Carol Kuhn, Jessica H. Borowsky, Eric Wiewiora, D. E. Shaw Research и Gregory R. Bowman. “SARS-CoV-2 Simulations Go Exascale to Capture Spike Opening and Reveal Cryptic Pockets Across the Proteome”. B: *Nature Chemistry* 13.7 (2021), с. 651—659. DOI: [10.1038/s41557-021-00707-0](https://doi.org/10.1038/s41557-021-00707-0).